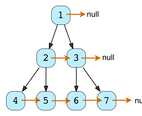
Solution: Connect Level Order Siblings

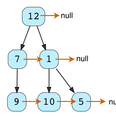
Problem Statement

Given a binary tree, connect each node with its level order successor. The last node of each level should point to a `null` node.

Example 1:



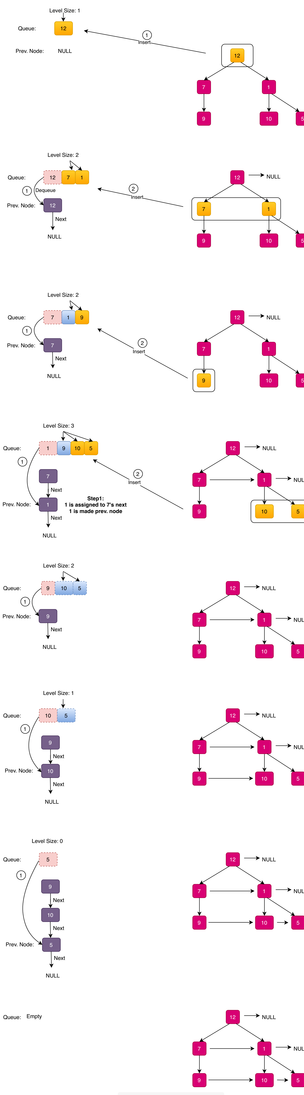
Example 2:



Solution

This problem follows the Binary Tree Level Order Traversal pattern. We can follow the same BFS approach. The only difference is that while traversing a level we will remember the previous node to connect it with the current node.

Here is the visual representation of the algorithm:



Connect Level Order Siblings

Code

Here is the code for this algorithm:

```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;
//   TreeNode next;

//   TreeNode(int x) {
//     val = x;
//     left = right = next = null;
//   }
// };

class Solution {
  public TreeNode connect(TreeNode root) {
    if (root == null)
      return root;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be $O(N)$, which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.